

Proyecto Bases de Datos - Programación:

Gestión de parking – **Los Panteras**



Proyecto realizado por:

- Jorge Alberto Fuentes Serrano
- Hugo Cruz Aranda

ÍNDICE

1. Introducción.....	3
2. Diagrama E/R Entidad-Relación	4
3. Paso a tablas	5
4. Proceso de Normalización	6
1. Tabla: aparcamiento	7
2. Tabla: clientes	8
3. Tabla: estancias.....	9
4. Tabla: turnos	10
5. Tabla: proveedores	10
6. Tabla: articulos_almacen.....	11
7. Tabla: empleados	12
8. Tabla: servicios_disponibles.....	13
9. Tabla: empleado_turno.....	13
10. Tabla: estancia_servicio	14
11. Tabla: vehiculo	14

1.Introducción

Contexto General: Se requiere diseñar una base de datos y desarrollar una aplicación de escritorio para informatizar la gestión de una cadena de tres aparcamientos situados en Calatayud, anexos a distintas sucursales del Centro Comercial "Los Panteras". Cada aparcamiento funciona de manera independiente, opera las 24 horas del día y tiene una capacidad máxima de hasta 600 vehículos.

Tipos de Usuarios y Control de Acceso: El sistema debe ser capaz de registrar y distinguir entre tres perfiles de clientes, aplicando diferentes reglas de negocio y facturación para cada uno:

- **Usuarios Normales:** Acceden mediante un ticket que registra la fecha, hora de entrada y matrícula. El coste es de 0,90 € la primera media hora, 0,60 € la segunda, y 0,90 € por cada media hora adicional. Si presentan un ticket de compra del Centro Comercial, obtienen 2 horas gratuitas. Tienen un límite de estancia de 24 horas (con sanción si se supera) y una multa de 24,04 € en caso de pérdida del ticket.
- **Usuarios Abonados:** Pagan una cuota mensual de 120,20 €. Acceden mediante una tarjeta identificativa que contiene su número de abonado, código del parking y número de plaza. Tienen una plaza fija asignada (el aparcamiento reserva el 20% de su capacidad total para ellos) y no tienen límite de estacionamiento.
- **Trabajadores del parking:** Se identifican con su propia tarjeta a la entrada y salida. Tienen características de acceso similares a los abonados, pero sin una plaza fija reservada.

Servicios Adicionales e Inventario: El parking ofrece servicios extra a los clientes. Al contratar cualquiera de ellos, el cliente recibe una hora de aparcamiento gratis:

- **Limpieza de vehículos:** Disponible de 09:00 a 22:00. Cuesta 3 € (solo exterior) o 5,41 € (interior y exterior).
- **Cambio de aceite:** Tarifa fija de 21,04 €.

El sistema debe vincular los servicios solicitados a la tarjeta/ticket del cliente para cobrar el total a la salida. Además, debe gestionar el inventario de los productos utilizados para estos servicios. Cuando el stock de un artículo en el almacén cae al 7% de su capacidad, el sistema debe gestionar automáticamente un pedido a los proveedores (almacenando datos como nombre, dirección, teléfono y tiempo de entrega).

Gestión de Personal y Turnos: La aplicación debe administrar una plantilla total de 16 trabajadores (6 guardias de seguridad y 10 empleados).

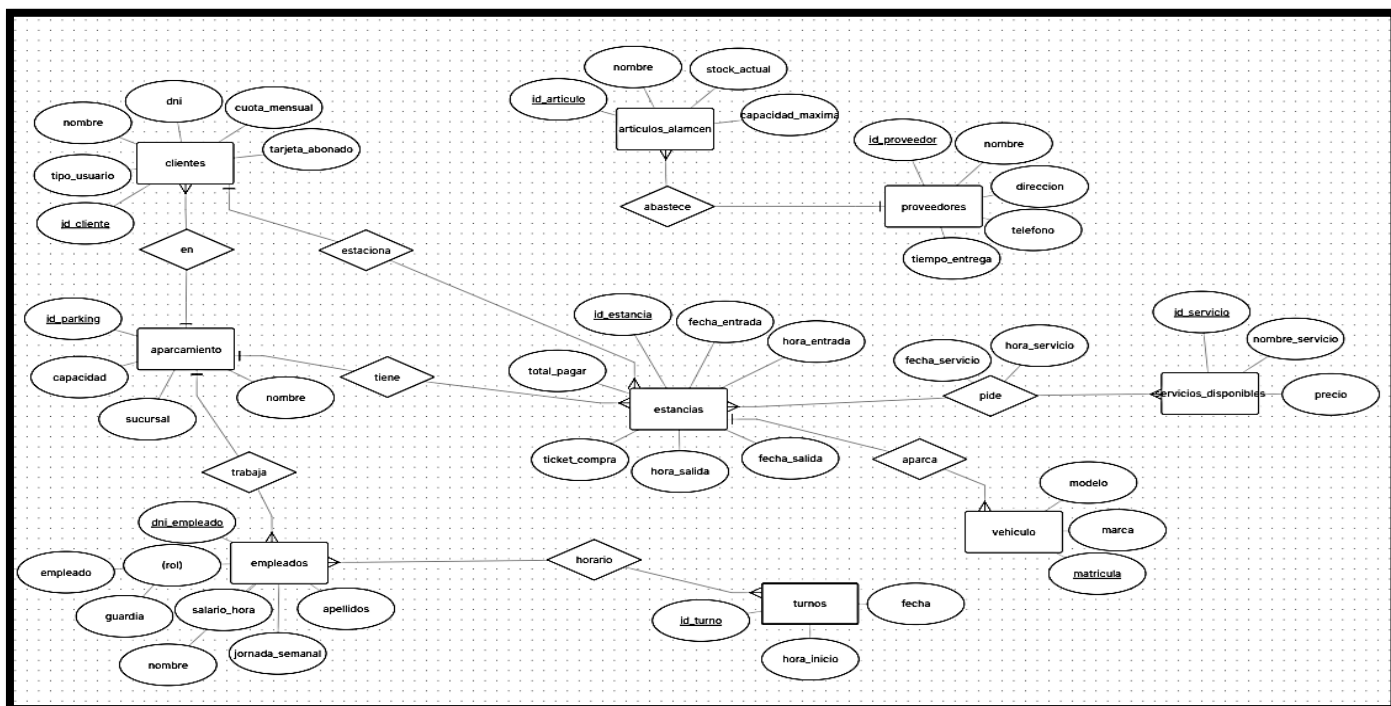
- Se organizan en turnos de 8 horas (compuestos siempre por 1 guardia y 2 empleados).
- Cada empleado está asignado a un único aparcamiento, cumple una jornada de 40 horas semanales y tiene un salario de 5,41 € por hora

2. Diagrama E/R Entidad-Relación

El Modelo Entidad/Relación (E/R) representa el esquema conceptual de nuestra base de datos, diseñado para capturar de forma visual y semántica la lógica de negocio de la cadena de aparcamientos "CC Los Panteras". Este diagrama nos ha permitido identificar las entidades principales del sistema, sus atributos y cómo interactúan entre sí antes de proceder a su implementación técnica.

Decisiones clave del diseño conceptual:

- **Núcleo de Operaciones:** La entidad central del modelo es **Estancias**. Funciona como el eje vertebrador que conecta a los clientes (ya sean abonados o de rotación), los vehículos que acceden al recinto y la sucursal (aparcamiento) específica donde se produce el estacionamiento.
- **Gestión de Servicios Adicionales:** Se ha modelado una relación de cardinalidad muchos a muchos (N:M) entre las estancias y los servicios disponibles (como lavado o revisión). Destaca el hecho de que esta relación ("pide") posee atributos propios (fecha y hora), lo que es vital para registrar el momento exacto en el que se solicita un servicio durante una estancia concreta.
- **Módulos de Gestión Interna:** Para mantener el sistema escalable, se han separado conceptualmente las operaciones internas en dos bloques independientes del flujo de clientes: la gestión de Recursos Humanos (relación entre **Empleados** y **Turnos**) y la gestión de Inventario (relación entre **Artículos del Almacén** y **Proveedores**).

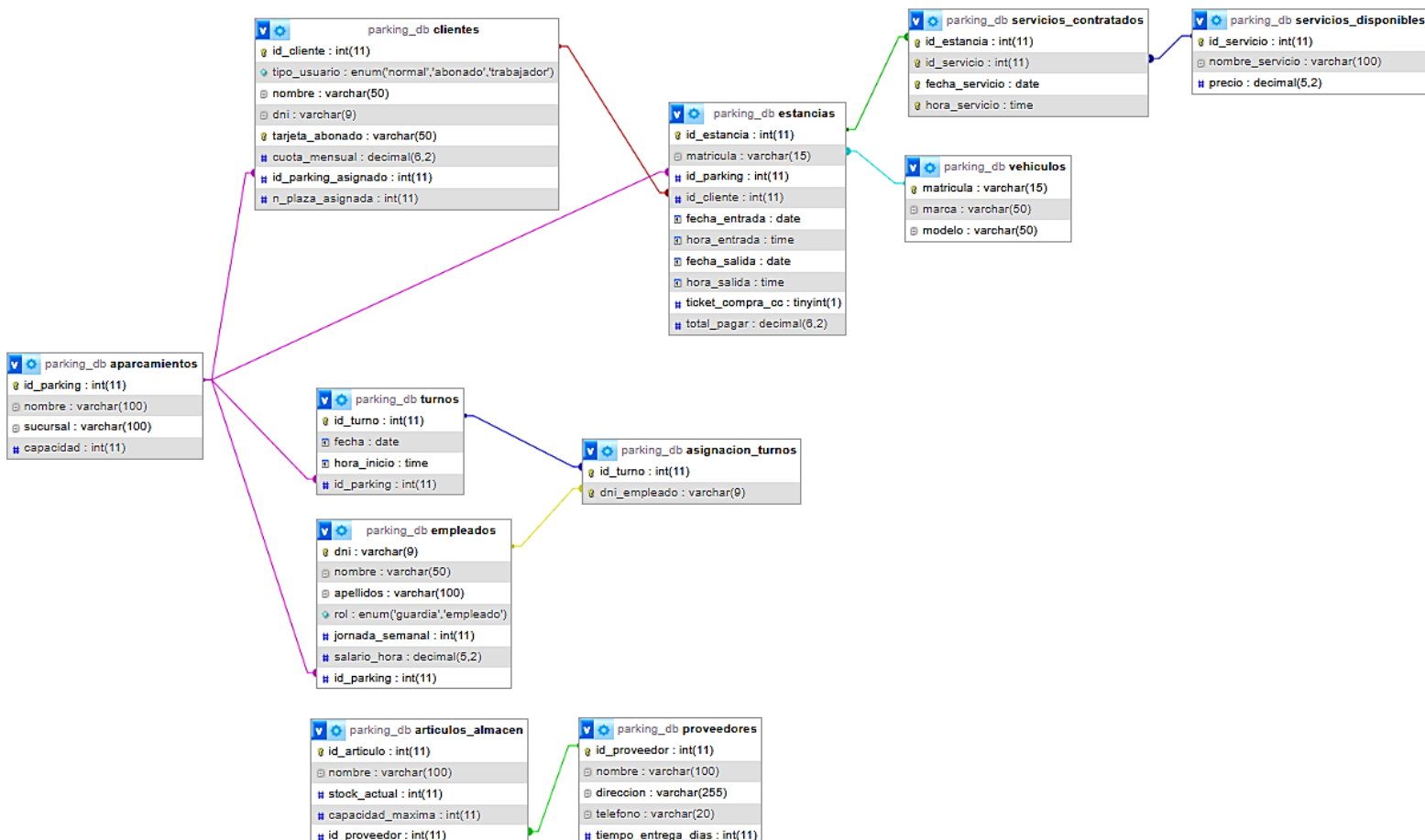


3. Paso a tablas

Una vez con el modelo E/R completo, pasamos a convertirlo al **modelo Relacional**.
Teniendo esta conversación logramos obtener una mejor visión de como se verá con la base de datos creada.

Reglas de transformación aplicadas:

- **Migración de Claves en Relaciones 1:N (Uno a Muchos):** En las relaciones de uno a muchos, la clave primaria de la entidad "fuerte" ha migrado como clave foránea a la entidad dependiente. Por ejemplo, el `id_parking` de la tabla `Aparcamientos` se propaga a las tablas `Cientes`, `Empleados` y `Estancias`, permitiendo segmentar toda la información por sucursal. De igual modo, el `id_proveedor` ha migrado a la tabla `Articulos_Almacen`.
- **Resolución de Relaciones N:M :** Como dicta la teoría relacional, las relaciones N:M del esquema E/R no pueden implementarse directamente. Por ello, creamos nuevas tablas intermedias que combinan las claves primarias de las entidades relacionadas:
 - `servicios_contratados` : Nace de la relación entre `Estancias` y `Servicios` . Además de heredar `id_estancia` e `id_servicio` como su clave primaria compuesta, también le incluimos los atributos que estaban en la relación.
 - `asignacion_turnos` : Nace de la relación entre `Empleados` y `Turnos` , vinculando el `dni_empleado` con el `id_turno` para llevar un control de los turnos de trabajo.



4. Proceso de Normalización

Para garantizar que el diseño de nuestra base de datos sea eficiente, evitar redundancia de información y prevenir anomalías en las operaciones de inserción, actualización y eliminación de datos, la hemos sometido al modelo relacional, aplicando las tres primeras Formas Normales (1FN, 2FN y 3FN). Este análisis asegura una correspondencia exacta con las tablas físicas implementadas en MySQL y los DAOs que gobiernan nuestra aplicación Java.

1. Primera Forma Normal (1FN): Atomicidad

- **Requisito:** Todos los atributos deben ser únicos, lo que significa que cada campo debe contener un único valor indivisible (atómico) y no pueden existir grupos repetitivos o listas dentro de una celda. Además, cada tabla debe disponer de una clave primaria (PK) única.
- **Aplicación en el sistema:** Todas las tablas cumplen con este principio de atomicidad. En la tabla **Empleados**, los datos como nombre, apellidos, puesto o salario_hora se almacenan en columnas independientes. Asimismo, las relaciones de muchos a muchos se han descompuesto en tablas intermedias normalizadas para evitar colecciones de datos dentro de una misma celda. Por ejemplo, cada servicio consumido se registra como una fila independiente en la tabla intermedia **Servicios_Contratados**, en lugar de almacenarse en un único campo de texto.

2. Segunda Forma Normal (2FN): Dependencia funcional completa

- **Requisito:** El modelo debe cumplir con la 1FN y, además, todos los atributos que no forman parte de la clave primaria deben depender de forma completa de toda la clave primaria, y no de una parte de ella (eliminación de dependencias parciales). Esto aplica especialmente a las tablas con claves primarias compuestas.
- **Aplicación en el sistema:** Las entidades con claves primarias simples (Aparcamientos, Empleados, Vehículos) cumplen la 2FN de manera automática. El análisis exhaustivo se centró en las tablas con claves compuestas:
 - **Asignacion_Turnos** (PK: **dni + id_turno**): Los atributos propios corresponden estrictamente a la asociación del empleado con su franja horaria. Los datos maestros del operario se quedan en Empleados y los del horario en Turnos, evitando que dependan únicamente del DNI dentro de esta tabla.
 - **Servicios_Contratados** (PK: **id_estancia + id_servicio**): Las columnas describen únicamente la contratación del servicio para una estancia específica. Los nombres y descripciones de los servicios dependen en exclusiva de la tabla maestra Servicios, resolviendo cualquier dependencia parcial.

3. Tercera Forma Normal (3FN): Eliminación de dependencias transitivas

- **Requisito:** El modelo debe cumplir con la 2FN y, además, ningún atributo no primario puede depender de forma transitiva de la clave primaria. Es decir, se deben eliminar las dependencias entre columnas que no sean claves.
- **Aplicación en el sistema:** El caso más crítico resuelto fue la gestión de los artículos de consumo del almacén y sus proveedores. Si hubiéramos guardado el teléfono o el contacto del proveedor dentro de la tabla de inventario ([Articulos_Almacen](#)), estaríamos rompiendo la 3FN, ya que el teléfono dependería del nombre del proveedor, y este a su vez del ID del artículo. Como solución, se aisló toda la información de contacto en la tabla independiente [Proveedores](#). En [Articulos_Almacen](#) únicamente se almacena el [id_proveedor](#) como clave foránea (FK). Si los datos cambian, se modifican en un único lugar sin afectar los registros de stock.

1. Tabla: aparcamiento

Campo	Tipo de Clave	Descripción
id_aparcamiento	PK	Identificador único del aparcamiento.
nombre	-	Nombre del recinto.
sucursal	-	Ubicación de la sucursal.
capacidad	-	Capacidad máxima de vehículos.

2. Tabla: clientes

Campo	Tipo de Clave	Descripción
id_cliente	PK	Identificador único del cliente.
dni	-	Documento de identidad.
nombre	-	Nombre del cliente.
tipo_usuario	-	Tipo (Abonado, Normal).
cuota_mensual	-	Cuota de abono si aplica.
tarjeta_abonado	-	Número de tarjeta del abonado.
id_aparcamiento	FK	Aparcamiento asignado.

3. Tabla: estancias

Campo	Tipo de Clave	Descripción
id_estancia	PK	Identificador de la visita.
fecha_entrada	-	Fecha de ingreso.
hora_entrada	-	Hora de ingreso.
fecha_salida	-	Fecha de finalización.
hora_salida	-	Hora de finalización.
ticket_compra	-	Código o estado del ticket.
total_pagar	-	Importe a abonar.
id_cliente	FK	Cliente asociado.
id_aparcamiento	FK	Aparcamiento utilizado.
matricula	FK	Vehículo estacionado.

4. Tabla: turnos

Campo	Tipo de Clave	Descripción
id_turno	PK	Identificador del turno.
fecha	-	Fecha del turno.
hora_inicio	-	Hora en la que comienza el turno.
id_parking		Que turnos corresponden a que parking

5. Tabla: proveedores

Campo	Tipo de Clave	Descripción
id_proveedor	PK	Identificador del proveedor.
nombre	-	Nombre de la empresa proveedora.
direccion	-	Dirección del proveedor.
telefono	-	Teléfono de contacto.
tiempo_entrega	-	Tiempo estimado de entrega.

6. Tabla: articulos_almacen

Campo	Tipo de Clave	Descripción
id_articulo	PK	Identificador del artículo.
nombre	-	Nombre del consumible.
stock_actual	-	Cantidad disponible.
capacidad_maxima	-	Límite de almacenaje.
id_proveedor	FK	Proveedor que lo suministra.

7. Tabla: empleados

Campo	Tipo de Clave	Descripción
dni	PK	Identificador único del empleado.
nombre	-	Nombre del empleado.
apellidos	-	Apellidos del empleado.
empleado	-	Rol o tipo de empleado.
guardia	-	Indica si realiza guardias (Booleano).
salario_hora	-	Salario por hora trabajada.
jornada_semanal	-	Horas de jornada a la semana.
id_aparcamiento	FK	Sucursal donde trabaja.

8. Tabla: servicios_disponibles

Campo	Tipo de Clave	Descripción
id_servicio	PK	Identificador del servicio.
nombre_servicio	-	Nombre del servicio.
precio	-	Coste del servicio.

9. Tabla: empleado_turno

Campo	Tipo de Clave	Descripción
id_empleado	PK, FK	Vinculado al empleado.
id_turno	PK, FK	Vinculado al turno asignado.

10. Tabla: estancia_servicio

Campo	Tipo de Clave	Descripción
id_estancia	PK, FK	Vinculado a la estancia.
id_servicio	PK, FK	Vinculado al servicio contratado.
fecha_servicio	-	Atributo de la relación (fecha).
hora_servicio	-	Atributo de la relación (hora).

11. Tabla: vehiculo

Campo	Tipo de Clave	Descripción
matricula	PK	Identificador único del vehículo.
marca	-	Marca del vehículo.
modelo	-	Modelo del vehículo.

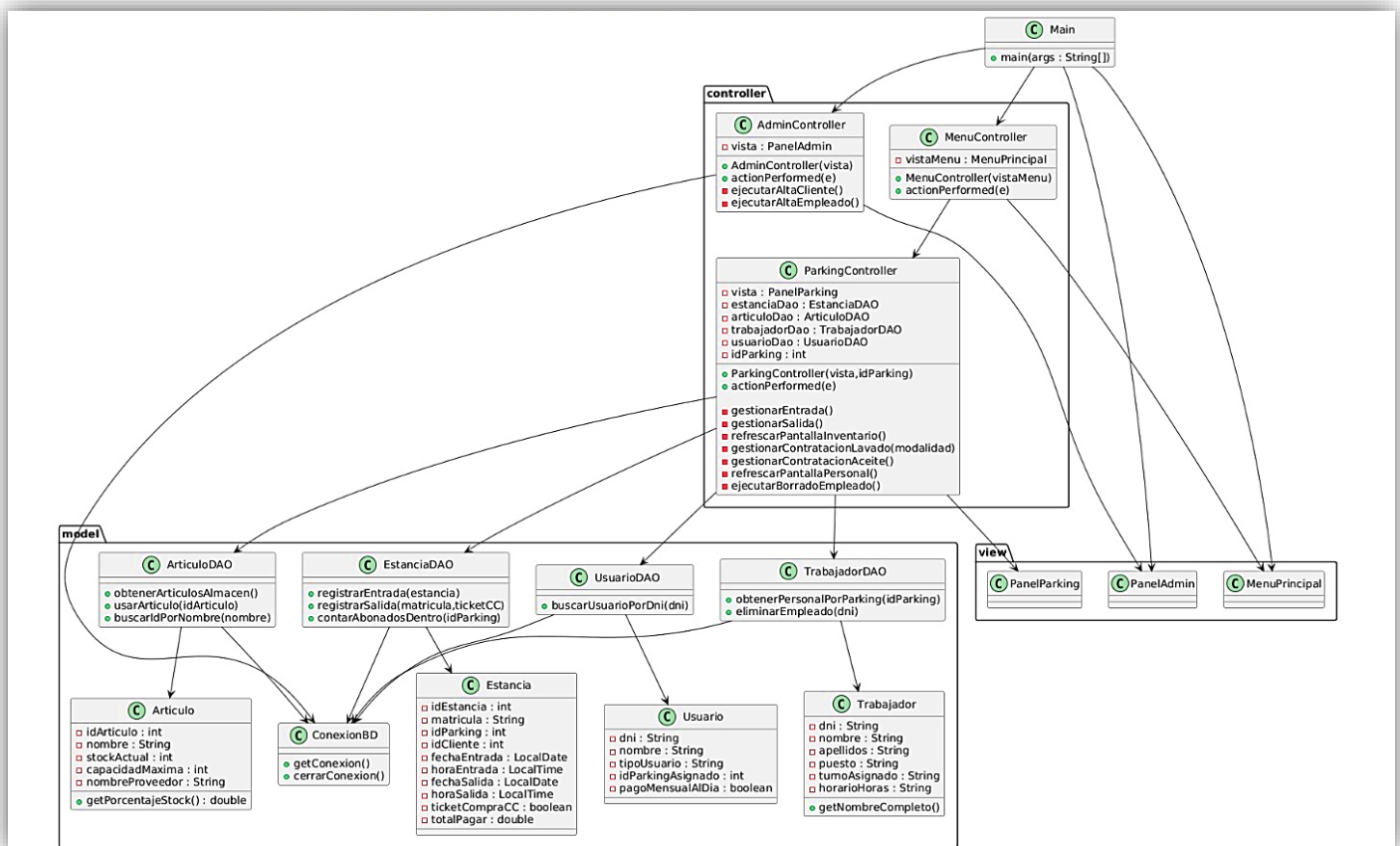
5.Arquitectura de Software y Diagrama de Clases UML

Para el desarrollo de la aplicación de gestión del parking se ha implementado de forma estricta el patrón de diseño **Modelo-Vista-Controlador (MVC)**. Este enfoque arquitectónico permite separar de manera clara la interfaz gráfica, la lógica de negocio y la persistencia de datos en la base de datos relacional MySQL, garantizando un código mantenible, escalable y con alta cohesión.

A continuación, se detalla la organización del sistema reflejada en el diagrama de clases unificado:

- **Capa del Modelo (model):** Contiene las clases entidad puras que representan los objetos de nuestro dominio de negocio, tales como Usuario, Estancia, Trabajador y Articulo, encapsulando sus atributos privados y métodos de acceso. Junto a ellas, se sitúan las clases de acceso a datos (**DAO**), como EstanciaDAO, ArticuloDAO, TrabajadorDAO y UsuarioDAO, encargadas de interactuar de forma aislada y directa con la clase auxiliar ConexionBD para ejecutar las consultas de persistencia y transacciones sobre MySQL.
- **Capa de la Vista (view):** Integrada por los formularios y componentes gráficos nativos del sistema desarrollados mediante la biblioteca javax.swing. Clases como PanelParking, PanelAdmin y MenuPrincipal actúan de forma pasiva, limitándose a pintar los elementos en la pantalla del operario y exponiendo getters limpios para que sus componentes sean accesibles por los controladores.
- **Capa del Controlador (controller):** Representa el cerebro operacional del software. Las clases ParkingController y AdminController implementan la interfaz ActionListener de Java para capturar los eventos y clics de los botones de las vistas. El controlador actúa como intermediario: recibe las acciones del usuario, solicita o modifica información a través de los métodos de los DAOs y actualiza dinámicamente el estado de los componentes visuales en tiempo real.

DIAGRAMA UML



En este diagrama de clases se ve perfectamente cómo hemos montado todo el ciclo de vida de los datos (el **CRUD**) que nos piden en la práctica para que la aplicación sea completa:

- **Create (Inserción):** Cuando un coche entra al parking, el programa crea una nueva estancia llamando al método `registrarEntrada(estancia)` dentro de `EstanciaDAO`. Lo mismo pasa en el panel de administración central con los métodos de alta del `AdminController` para meter nuevos clientes o trabajadores.
- **Read (Consulta):** Las lecturas de la base de datos se hacen solas de forma automática en cuanto se abren las pantallas. Para eso usamos métodos de consulta masiva que rellenan las tablas y listas de la interfaz, como `obtenerArticulosAlmacen()` para ver el stock del almacén o `obtenerPersonalPorParking(idParking)` para cargar el cuadrante de los operarios.
- **Update (Modificación):** Las actualizaciones se ejecutan principalmente cuando un vehículo va a salir del parking. En ese momento, el sistema calcula el tiempo que ha estado dentro, mira si tiene descuento del centro comercial y modifica el registro de la estancia llamando a `registrarSalida(matricula, ticketCC)` para cerrarla y poner el precio final.

- **Delete (Eliminación):** Para completar las cuatro operaciones del CRUD y que no nos faltase ninguna en la entrega, añadimos una opción de borrado físico en el módulo de personal. Desde la pantalla se puede seleccionar un trabajador y, a través del método `eliminarEmpleado(dni)` en `TrabajadorDAO`, borrarlo definitivamente de la base de datos usando su DNI como clave. Así el flujo queda bien cerrado y cumple con todo lo exigido en las tres asignaturas.